KEYGUARD

Digital Vehicle Access Security Laboratory

Product requirements • Technical architecture • MVP scope • Security validation • Implementation roadmap • Claude Code handoff

Version 1.0 | November 2026 | Status: proposed specification, not implemented functionality

Vision: Make digital vehicle authentication understandable, testable, and measurable through a polished interactive simulator backed by actual cryptographic verification and reproducible adversarial tests.

Design principle: The interface displays decisions made by the backend. Never substitute frontend animations or canned success messages for actual authorization results.

Safety boundary: Fictional vehicles and simulated credentials only. No real Tesla integration, RF attacks, proprietary protocol cloning, or real vehicle control.

1. Executive summary and scope

KeyGuard is a desktop-first responsive web application combining a digital-key authentication simulator, security experiment laboratory, protocol inspector, and event analytics dashboard. Its first release demonstrates ECDSA P-256 challenge-response authentication, credential revocation, freshness and replay protection, and a deliberately vulnerable replay comparison in isolated lab state.

Dimension	Decision
Audience	Cybersecurity students, engineers, educators, and vehicle technology enthusiasts
Platform	Next.js/TypeScript frontend and Python/FastAPI backend
Storage	SQLite for local development; PostgreSQL when hosted
Initial key	Simulated smartphone digital key
Target duration	8–10 weeks at approximately 10–15 hours/week (estimate)
Primary deliverable	Working end-to-end demo plus reproducible tests and documented limitations

Goals

- Show real signature verification and authorization checks.
- Allow users to inspect authentication stages and denial reasons.
- Demonstrate replay, invalid signature, expiration, cross-vehicle, and revocation tests.
- Publish reproducible security evidence and a polished accessible interface.

Explicit non-goals

- Unlocking or controlling real vehicles or handling actual vehicle credentials.
- Replicating Tesla proprietary phone-key, NFC, BLE, or key-fob protocols.
- Claiming resistance to physical relay attacks or proof of physical proximity.
- Production-grade automotive certification or a universal security score.

2. Users and product requirements

Persona	Need	Successful outcome
Security student	Understand protocol and test attacks	Explains why a replay fails in hardened mode
Security engineer	Repeatable authentication validation	Runs test matrix and inspects evidence
Vehicle enthusiast	Understand digital access security	Completes guided flow without crypto expertise

Primary user journeys

- Normal: create lab → enroll public key → request unlock challenge → sign challenge locally → server verifies → vehicle unlocks → inspector explains each step.

- Replay: obtain a previously valid response in an isolated experiment → resubmit → vulnerable lab verifier may accept; hardened verifier rejects consumed challenge.
- Revocation: enroll two keys → revoke one → valid active key authenticates; revoked key is denied even with a valid signature.

3. Prioritized MVP backlog

ID	Priority	Feature	Acceptance condition
F01	P0	Garage dashboard	Vehicle state reflects backend-authorized changes
F02	P0	Key enrollment	Client generates key pair; server stores public key only
F03	P0	Challenge-response	Valid key succeeds; invalid credentials denied
F04	P0	Protocol inspector	Shows challenge, signature verification and decision stages
F05	P0	Replay lab	Reproducible vulnerable/hardened comparison
F06	P0	Revocation	Revoked credential cannot authenticate
F07	P0	Automated tests	Required cases run in CI
F08	P1	Security event dashboard	Filter and inspect authentication events
F09	P1	Tampering scenarios	Demonstrate invalid-signature rejection
F10	P2	Key card/fob simulations	Additional fictional credential models
F11	P2	Proximity visualization	Clearly conceptual; no physical proximity claim
F12	P2	Accounts and saved labs	Secure persisted history and access control

Scope rule: Do not begin P1/P2 until the seven P0 features work end-to-end and required security tests pass.

4. System architecture

```
Browser (Next.js + TypeScript)
  |-- Garage / Digital Keys / Inspector / Attack Lab / Analytics
  |-- Web Crypto API: generate non-extractable key + sign challenge
  |
  +---- HTTPS JSON API ----> FastAPI backend
                              |-- Lab/session authorization
                              |-- Authentication engine
                              |-- Isolated attack scenario engine
                              |-- Event/audit logging
                              +---- SQLite / PostgreSQL

pytest + Playwright + GitHub Actions -> regression evidence
```

Component ownership

- Frontend: accessible views, client key generation and signing, user journeys, status and explanations.
- API: request validation, lab isolation, rate limits, authenticated session authorization, structured responses.

- Authentication engine: challenge issuance, public-key signature verification, key authorization, expiry and atomic consumption.
- Attack lab: predefined scenarios operating only on disposable, separate state; never imported into hardened production authorization path.
- Persistence: vehicles, public credentials, challenges, security events and lab results.

Critical trust boundaries

- Never trust client-supplied success flags or client-controlled vehicle state.
- Private keys remain client-side; the backend receives only public keys and signatures.
- Browser non-extractable keys are not equivalent to hardware-backed automotive credentials.
- Each lab requires a scoped authorization token; users must not read or mutate other labs.
- Do not log private keys, raw session secrets, or sensitive credential material.

5. Authentication protocol specification

Use established libraries with ECDSA P-256 and SHA-256. This is an educational protocol design, not a reproduction of any manufacturer protocol. Choose one canonical, versioned encoding for signed messages; document its exact bytes and test vectors before implementing client/server interoperability.

Protocol sequence

- 1. Client creates a P-256 key pair and registers its public key to a fictional vehicle in its authorized lab.
- 2. Client requests an unlock challenge for a specific vehicle and key.
- 3. Server creates a cryptographically random nonce and challenge ID, associates action, vehicle, key and expiration, then stores it as unused.
- 4. Client signs the canonical message containing protocol version, vehicle ID, key ID, challenge ID, nonce, action, and expiry.
- 5. Server verifies key status, vehicle/action binding, challenge expiry, signature validity and unconsumed status.
- 6. Server atomically marks the challenge consumed and commits the authorized vehicle state transition.
- 7. API returns a structured decision and sanitized trace for the inspector.

Verification invariants

- Only enrolled active keys authorized for the requested vehicle/action may authenticate.
- A challenge may be used successfully at most once, including under concurrent requests.
- Expired, malformed, foreign-vehicle, action-mismatched and previously consumed challenges fail closed.
- Signed data is domain-separated and unambiguously encoded; never concatenate fields without framing.
- Cryptographic validity is necessary but not sufficient for authorization.
- Successful signature verification does not prove physical proximity; relay remains out of scope.

6. Data model and API contract

Entity	Suggested fields
vehicles	id, lab_id, name, lock_state, created_at
digital_keys	id, lab_id, vehicle_id, public_key, status, created_at, revoked_at
challenges	id, lab_id, vehicle_id, key_id, nonce, action, expires_at, consumed_at
security_events	id, lab_id, vehicle_id, key_id, event_type, outcome, reason, created_at
lab_runs	id, lab_id, scenario, mode, expected_result, actual_result, created_at

Method	Endpoint	Purpose
POST	/api/labs	Create scoped disposable lab
GET	/api/labs/{id}/vehicles	Read virtual vehicle state
POST	/api/labs/{id}/keys	Enroll public key
POST	/api/labs/{id}/keys/{keyId}/revoke	Revoke credential
POST	/api/labs/{id}/challenges	Issue scoped challenge
POST	/api/labs/{id}/authenticate	Verify signature and authorize
GET	/api/labs/{id}/events	Read sanitized event history
POST	/api/labs/{id}/scenarios/{scenario}	Run predefined lab scenario

API rules: Require lab-scoped authorization, validate all request schemas, apply rate limits and TTL cleanup, use opaque identifiers, and return stable machine-readable error codes. Define request/response JSON schemas during implementation; these are proposed endpoints, not a finalized OpenAPI contract.

7. Threat model and security validation

Protected assets

- Vehicle lock state and authorization decisions; enrolled public-key registry and status; challenge freshness; lab session boundaries; integrity of security test results.

Threats within scope

- Replaying a previously valid authentication response.
- Submitting an incorrect or tampered signature.
- Using revoked, expired or unknown credentials.
- Cross-vehicle and cross-action credential confusion.
- Double-spending a challenge via simultaneous requests.
- Accessing another user’s lab through guessed identifiers.

Mandatory hardened test matrix

Scenario	Expected result
Active key + fresh challenge	Allow
Incorrect signature	Deny
Revoked key	Deny
Expired challenge	Deny
Consumed challenge replay	Deny
Wrong vehicle	Deny
Wrong requested action	Deny
Two concurrent submissions	At most one succeeds
Unknown key	Deny
Malformed payload	Reject safely
Cross-lab request	Deny

Each test records scenario ID, expected decision, observed decision, failure reason and timing. Distinguish security failure (unauthorized acceptance), functional failure (valid request denied), and infrastructure failure (untrustworthy/incomplete test). Use an atomic transaction or conditional update for challenge consumption.

8. User experience specification

Screen	Required functionality
Garage	Virtual vehicle, lock state, key status, request unlock/lock, latest event
Digital Keys	Enroll, inspect public-key metadata, revoke credentials
Authentication Inspector	Stepwise challenge, signature, freshness and authorization timeline
Attack Lab	Scenario picker, vulnerable/hardened comparison, expected vs actual
Security Analytics	Searchable event log, pass/fail counts and failure categories

Visual direction: desktop-first, dark automotive-inspired styling, restrained colors, readable typography, subtle vehicle transitions, clear success/denial states, accessible labels and keyboard navigation. Avoid presenting UI-only interactions as proof of real cryptographic security.

9. Recommended stack and repository

Layer	Choice
Web	Next.js, TypeScript, Tailwind CSS, optional Framer Motion
API	Python, FastAPI, Pydantic
Crypto	Web Crypto API client; vetted Python cryptography package server
Database	SQLite local; PostgreSQL for hosted persistent sessions
Tests	pytest, Playwright; concurrency and negative-path tests
CI	GitHub Actions
Hosting	Vercel frontend plus independently hosted API, after hardening

```
keyguard/
  apps/web/app/{garage,keys,inspector,lab,analytics}/
  apps/web/components/
  apps/web/lib/{api.ts,crypto.ts}
  services/api/app/{routes,models,security,lab}/
  services/api/tests/
  docs/{PRD,ARCHITECTURE,THREAT_MODEL,SECURITY_TESTS,DECISIONS}.md
  .github/workflows/
  docker-compose.yml
  README.md
```

10. Eight-week implementation plan

Week	Work	Exit deliverable
1	PRD, threat model, repo scaffolding, Next.js/FastAPI, Garage shell	Running frontend/backend
2	Browser key generation, enrollment, vehicle/key models	Working enrollment
3	Challenge issuance, signing, verification, expiry, atomic consume	Authenticated unlock
4	Negative tests, revocation, binding checks, concurrency	Passing core test suite
5	Isolated vulnerable verifier and replay comparison	Reproducible replay lab
6	Protocol timeline, explanations, vehicle animations	Interactive inspector
7	Events, metrics, GitHub Actions, limits, cleanup	CI and analytics
8	Usability sessions, bug fixes, docs, deployment and demo	Portfolio-ready MVP

Schedule is an estimate. Security correctness takes precedence over deadline or visual polish.

11. Success metrics and release criteria

Metric	Proposed target
Mandatory security tests	100% passing before release
Unauthorized unlocks in required cases	0
Usability task completion	At least 80% across 5–8 initial testers
User-rated clarity	Average at least 4/5
Local median simulated authentication response	Under 300 ms, excluding real wireless latency

These are targets, not achieved outcomes. A passing curated test suite does not prove universal security. Report test counts, coverage boundaries and unresolved risks.

Release gates

- All seven P0 features work end-to-end and the backend is authoritative.
- No known unauthorized unlock in the mandatory test suite; concurrency and cross-lab isolation verified.
- Attack lab is isolated from the hardened authentication path.
- Automated tests run in CI and failures block release.
- Threat model, architecture, test evidence and limitations are documented.
- Hosted demo has session isolation, resource limits, and event retention controls.

12. Risks, assumptions and open decisions

Risk or decision	Guidance
Crypto interoperability	Agree on canonical signed bytes and test vectors before UI integration
Concurrency	Verify atomic challenge consumption in the chosen database
Browser key persistence	Define whether key material persists across refreshes; document lifecycle
Demo misuse	Use fixed scenarios, isolated state, strict input validation and rate limits
Privacy	Do not collect real vehicle data or private keys
Scope creep	No BLE/NFC/fob/relay features in MVP
Hosting	Local-first; deploy only after security controls and tests
Auth/session design	Specify lab-token issuance, expiry and storage before public release

13. Claude Code implementation handoff

Operating instructions for coding assistant: Treat this PDF as a proposed specification. Before writing major code, identify ambiguities and propose a short implementation decision record. Build vertical slices and show test evidence at every stage. Never report unimplemented features as complete.

Development principles

- Start with a local runnable monorepo and minimal automated checks.
- Implement the security protocol and tests before visualizing successful unlocks.
- Use standard libraries and documented APIs; do not invent cryptographic primitives.
- Keep vulnerable educational logic physically separated from hardened authorization logic.
- Add typed request/response schemas, structured error codes and secure logging.
- Do not add real Tesla APIs, key fob radios, Bluetooth attacks, or proprietary integration.
- Maintain docs/DECISIONS.md for choices, assumptions and tradeoffs.
- After each milestone: run tests, report what passed/failed, update README and list remaining risks.

Suggested initial prompt for Claude Code

Copy the following instruction into Claude Code alongside this PDF (or paste its extracted text into the repository):

```
You are my senior full-stack and security engineering partner.
Implement KeyGuard using the attached product specification.
First: review the requirements, identify ambiguities, propose
a minimal architecture and a sprint-1 task list. Do not code
until we agree on the protocol encoding and lab authorization.
Then implement small, testable vertical slices. Prioritize
backend-verified ECDSA authentication, challenge freshness,
atomic consumption, key revocation, and isolated replay tests.
Never claim a security property without executable evidence.
Do not integrate with real vehicles or proprietary protocols.
For each change: explain files, tests run, results and next steps.
```

First sprint acceptance checklist

- Repository contains working Next.js and FastAPI applications with setup instructions.
- A documented threat model and signed-message encoding decision exist.
- A standalone test generates a key pair, signs a challenge, verifies it, and rejects replay.
- Garage interface renders with clearly marked mock state until backend integration.
- CI runs the initial test suite.

14. Portfolio presentation

The public README should explain the problem, architecture, threat model, live demonstration, a successful authentication, a replay rejection, test evidence, and limitations within the first minute. Include a short video or GIF of the UI, but distinguish visualization from verified security behavior.

Future resume language (use only after implementation): Built an interactive digital vehicle-access security laboratory implementing ECDSA challenge-response authentication, credential revocation and replay

protection; developed isolated adversarial regression tests and an authentication inspector to explain access decisions.

Next product: PrivacyLens, a separate privacy engineering project focused on user-imported device activity reports, permissions and network-data analysis. Do not expand KeyGuard into PrivacyLens during MVP development.